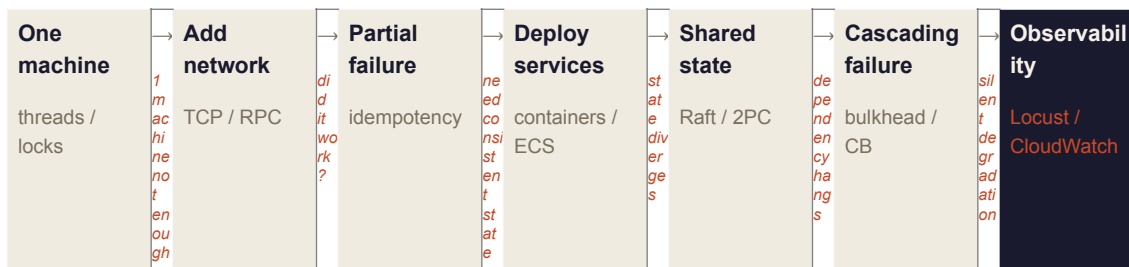


What I've Found Valuable So Far

Emily Chen · M.S. Computer Science, Northeastern

The through-line of this course is a single escalating question: *how do we run code reliably when one machine is never enough?* One machine is not enough, so we add a network. But network calls can fail ambiguously — you cannot tell if the work was done or not. So we need consensus and transactions. We package services into containers so they deploy consistently. But multiple replicas mean shared state can diverge. And when a downstream dependency hangs, it silently drains your own thread pool. Every topic is a direct answer to the problem the previous answer created.

The causal chain — each solution creates the next problem:



1 — One Machine: Threads Share Memory

Every thread gets its own stack, so local variables are safe. But all threads share the heap — shared mutable state requires coordination. The entire toolkit of mutexes, thread pools, and thread-safe collections flows from one fact: if two threads can touch the same memory location, one must wait. This is the root problem, and every later topic is a variation of it at larger scale.

↓ *one machine isn't enough → distribute across many*

2 — Adding a Network: The Partial Failure Problem

Once you move to multiple machines, the only communication path is the network — and it is unreliable. The key insight is that a local call has two outcomes: success or error. A network call has three: success, server never received it, or server did the work but the response was lost. The last two look identical from the caller's side — just silence. This is partial failure: if you retry, you risk doing the work twice. If you don't, you risk missing it. The question 'did it actually work?' becomes genuinely unanswerable, and everything in distributed systems is an attempt to answer it.

↓ *now we need to deploy and scale these services consistently*

3 — Containers and Orchestration: Consistent Deployment

Microservices create an operational problem: how do you run and scale dozens of services consistently? Containers solve packaging (image = blueprint, container = running instance). ECS solves orchestration. Fargate removes server management entirely. Terraform ties it together declaratively — describe the desired end state, and the tool computes what to create or destroy. In HW6, the entire cluster, ALB, subnets, and auto-scaling policies lived in one config file and were reproducible from scratch in minutes.

↓ *multiple replicas mean shared state can diverge*

4 — Distributed State: Consensus and Transactions

Partial failure makes distributed agreement hard. Two solutions address this from different angles. Raft elects one leader to serialize writes, replicating log entries to a majority before committing — ensuring no committed entry is ever lost across elections. Two-Phase Commit ensures cross-service transactions are atomic: all nodes commit or all abort together. The CAP theorem frames the underlying trade-off: since network partitions are unavoidable, the real choice is always between consistency and availability.

Raft and 2PC are both answers to the partial failure problem from Section 2 — the moment you cannot tell if a remote operation succeeded, you need machinery to enforce agreement across nodes.

↓ *a hung downstream dependency can silently drain your thread pool*

5 — Resilience: Bulkheads and Circuit Breakers

The danger is not raw traffic — it is a slow downstream service consuming your own threads while its callers wait. Without limits, one hung dependency fills your entire thread pool and your service appears down even though it is healthy. The bulkhead pattern assigns a fixed thread budget per downstream dependency, isolating the failure. The circuit breaker stops calling a failing service entirely and returns a fast fallback, giving it time to recover. The goal is graceful degradation: bound the blast radius, not prevent all failures.

↓ *how do you know when any of this is actually working?*

6 — Observability: HW6 as a Complete Case Study

HW6 made every prior concept measurable. A CPU-bound Go service on Fargate was load-tested with Locust while CloudWatch tracked CPU and memory. Memory stayed flat; CPU climbed linearly — sufficient to diagnose the bottleneck without guessing. Little's Law ($N = \lambda \times W$) explained why tail latencies blew out before the median moved: as latency W grew, in-flight requests N ballooned. The most transferable lesson came from a Piazza classmate: running Locust with zero wait time creates unbounded demand — new capacity is consumed immediately and auto-scaling never stabilizes. Client behavior is as much a design parameter as server configuration.

HW6 is where the chain closes: the concurrency model (§1) explains Go's burst resilience, the container stack (§3) is what CloudWatch observes, and Little's Law connects network latency (§2) to the auto-scaling policy.